

- Description

This module manages a smart warehouse automated guided vehicle (AGV) system, ensuring that:

1. **AGVs navigate efficiently within the warehouse by dynamically adjusting their paths to avoid congestion.**
2. **If a collision risk is detected, the AGV slows down or reroutes to prevent accidents.**
3. **If multiple AGVs are idle, task distribution is optimized to balance workload.**
4. **A log of all AGV movements, path adjustments, and task assignments is maintained for operational efficiency.**

This system helps optimize warehouse logistics, reduce delays, and improve autonomous vehicle coordination.

- Specification

module SmartAGVPathOptimizationEx

var currentPosition: integer

var isMoving: boolean

var log: sequence of string

process moveTo(newPosition: integer)

functional scenario form:

newPosition - currentPosition = a && isMoving = true, currentPosition = newPosition, log = log $\oplus$ ("AGV moved from Position " + currentPosition + " to " + newPosition), AGV moved successfully.

||

newPosition = currentPosition && log = log $\oplus$ ("AGV already at Position " + newPosition), No movement needed.

end process

process stopAGV()

functional scenario form:

AGV stopped successfully.

end process

end module

- Java Code

```
package code;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
/**
 * c125_SmartAGVSystem
 *
 * Single-file Smart AGV simulation with:
```

```

* - Core AGV model (keeps your moveTo/stop/printLog API and console me
ssages)
* - Linear topology (integer position line) + optional 2D projection h
elper
* - Battery + health model
* - Job/Task queue with simple scheduler (FCFS, priority bias, chargin
g rule)
* - Path planner (1D incremental moves) & collision reservation (time-
slot grid)
* - Sensors (position, battery, load)
* - Controller (rate limiting, soft clamping on absurd jumps)
* - Alerts & Logger & Stats
* - Scenarios (VT/FT/Z3/UVT) – UVT uses exactly your provided sequence
for agv2
* - Runner + CLI: MODE=VT|FT|Z3|UVT, SEED=<long>, LOG=on|off, AGVS=2..
16, LINE_LEN=200
*
* Notes:
* - Positions are integers on a line [0..LINE_LEN]; out-of-
range will be clamped (alert logged).
* - moveTo(newPosition) follows your original: if equal -> "already th
ere"; otherwise print & append log.
* - Scheduler only used by internal demo tasks; your VT/FT/Z3/UVT stil
l call moveTo directly.
* - CollisionGuard reserves (position,time) slots; in direct calls we
still check and warn.
* - Battery drains per unit distance & idle; automatic charging task i
nserted if too low.
* - All console prints keep your style for AGV operations; extra diagn
ostics go via logger.
*/
public class c125_SmartAGVSystem {
    /* ===== Alerts & Logger ===== */
    enum AlertType {
        INFO, WARN, ERROR,
        NEGATIVE_MOVE, OOB_POSITION, COLLISION_RISK, BATTERY_LOW, SENSO
R_FAULT,
        SCHEDULER_ENQUEUE, SCHEDULER_DISPATCH, PLANNER_REPLAN, RESERVAT
ION_CLASH
    }
    static final class Alert {
        final AlertType type;
        final String msg;
        final long ts;

```

```

        Alert(AlertType t, String m) { this.type = t; this.msg = m; this.ts = System.currentTimeMillis(); }
        @Override public String toString() { return "[" + type + "]" + msg + " @" + ts; }
    }
    static final class SystemLogger {
        final List<String> lines = new ArrayList<>();
        final List<Alert> alerts = new ArrayList<>();
        boolean console = true;
        void setConsole(boolean on) { console = on; }
        void log(String s) { lines.add(s); if (console) System.out.println(s); }
        void alert(AlertType t, String m) {
            Alert a = new Alert(t, m);
            alerts.add(a);
            if (console) System.out.println("ALERT: " + a);
        }
        void summary() {
            log("\n=== Logger Summary ===");
            log("Lines=" + lines.size() + ", Alerts=" + alerts.size());
            Map<AlertType, Long> cnt = new EnumMap<>(AlertType.class);
            for (Alert a : alerts) cnt.put(a.type, cnt.getOrDefault(a.type, 0L) + 1);
            for (AlertType t : AlertType.values()) log(" - " + t + ": " + cnt.getOrDefault(t, 0L));
        }
    }
    /* ===== Stats ===== */
    static final class Stats {
        long moves;
        long stops;
        long totalDistance;
        long replans;
        long reservations;
        long reservationClashes;
        long batteryCharges;
        long schedulerEnq;
        long schedulerDispatch;
        void print(SystemLogger L) {
            L.log("\n=== AGV Stats ===");
            L.log("Moves=" + moves + ", Stops=" + stops + ", Distance=" + totalDistance);
            L.log("Replans=" + replans + ", Reservations=" + reservations + ", Clashes=" + reservationClashes);
        }
    }

```

```

        L.log("AutoCharges=" + batteryCharges);
        L.log("Scheduler enq=" + schedulerEnq + ", dispatch=" + schedulerDispatch);
    }
}

/* ===== Grid / Topology helpers ===== */
static final class Line {
    final int length;
    Line(int length) { this.length = Math.max(1, length); }
    int clamp(int p) { return Math.max(0, Math.min(length, p)); }
    int distance(int a, int b) { return Math.abs(a - b); }
    // Optional: 2D projection just for pretty labeling (not required by logic)
    int[] toXY(int p, int width) {
        width = Math.max(1, width);
        int y = p / width;
        int x = p % width;
        return new int[]{x, y};
    }
}

/* ===== Reservation Table (collision/time-slot) ===== */
static final class ReservationTable {
    // Key: time slot; Value: set of occupied integer positions on the line
    final Map<Long, Set<Integer>> occ = new ConcurrentHashMap<>();
    final SystemLogger L;
    final Stats stats;
    ReservationTable(SystemLogger L, Stats stats) { this.L = L; this.stats = stats; }
    boolean tryReserve(long t, int pos, String agvId) {
        occ.computeIfAbsent(t, k -> Collections.synchronizedSet(new HashSet<>()));
        Set<Integer> s = occ.get(t);
        synchronized (s) {
            if (s.contains(pos)) {
                L.alert(AlertType.RESERVATION_CLASH, "t=" + t + " pos=" + pos + " by " + agvId);
                stats.reservationClashes++;
                return false;
            }
            s.add(pos);
            stats.reservations++;
            return true;
        }
    }
}

```

```

    }
}

/* ===== Battery & Health ===== */
static final class Battery {
    double level;          // 0..100
    final double idleDrainPerTick;
    final double moveDrainPerUnit;
    Battery(double init, double idleDrain, double moveDrain) {
        this.level = Math.max(0, Math.min(100, init));
        this.idleDrainPerTick = Math.max(0, idleDrain);
        this.moveDrainPerUnit = Math.max(0, moveDrain);
    }
    void drainIdle() { level = Math.max(0, level - idleDrainPerTick); }
    void drainMove(int distance) { level = Math.max(0, level - moveDrainPerUnit * Math.max(0, distance)); }
    void chargeTo(double v) { level = Math.max(0, Math.min(100, v)); }
    boolean low() { return level < 15.0; }
}

/* ===== Sensors ===== */
interface Sensor<T> { T read(); }
static final class PositionSensor implements Sensor<Integer> {
    final AGV agv;
    PositionSensor(AGV a) { this.agv = a; }
    @Override public Integer read() { return agv.currentPosition; }
}
static final class BatterySensor implements Sensor<Double> {
    final Battery b;
    BatterySensor(Battery b) { this.b = b; }
    @Override public Double read() { return b.level; }
}
static final class LoadSensor implements Sensor<Boolean> {
    final Random rnd;
    LoadSensor(long seed) { this.rnd = new Random(seed); }
    @Override public Boolean read() { return rnd.nextDouble() < 0.5; }
}

/* ===== Tasks & Planner ===== */
enum TaskType { MOVE, CHARGE }
static final class Task {
    final TaskType type;
    final int targetPos;    // for MOVE
}

```

```

        final int priority;        // Lower value => higher priority
        final String tag;
        Task(TaskType type, int targetPos, int priority, String tag) {
            this.type = type; this.targetPos = targetPos; this.priority
= priority; this.tag = tag;
        }
        @Override public String toString() { return type + (type==TaskT
ype.MOVE?("("+targetPos+")"): "") + "#p"+priority + (tag==null?"":["+ta
g+"]"); }
    }
    static final class Planner {
        final Line line;
        final ReservationTable table;
        final SystemLogger L;
        final Stats stats;
        Planner(Line line, ReservationTable table, SystemLogger L, Stat
s stats) {
            this.line = line; this.table = table; this.L = L; this.stat
s = stats;
        }
        /**
         * 1D plan: straight step-by-
step list of intermediate positions (exclusive of start, inclusive of e
nd).
         * Return empty if already there.
         */
        List<Integer> plan(int from, int to) {
            to = line.clamp(to);
            if (from == to) return Collections.emptyList();
            List<Integer> path = new ArrayList<>();
            int step = (to > from) ? 1 : -1;
            for (int p = from + step; p != to + step; p += step) path.a
dd(p);
            return path;
        }
        /**
         * Try to reserve time slots for the path; if fails, we try a s
imple wait-then-reserve replan.
         */
        List<long[]> reserve(List<Integer> path, long startTime, String
agvId) {
            List<long[]> times = new ArrayList<>();
            long t = startTime;
            for (int pos : path) {

```

```

        boolean ok = table.tryReserve(t, pos, agvId);
        if (!ok) {
            // simple backoff: wait 1 time unit, mark a replan
            (shift all following by +1)
            L.alert(AlertType.PLANNER_REPLAN, "Delayed step @t="
" + t + " pos=" + pos + " for " + agvId);
            stats.replans++;
            t++;
            ok = table.tryReserve(t, pos, agvId);
            if (!ok) {
                // as fallback, wait 2 more
                t += 2;
                table.tryReserve(t, pos, agvId);
                stats.replans++;
            }
        }
        times.add(new long[]{t, pos});
        t++;
    }
    return times;
}

}

/* ===== Scheduler ===== */
static final class Scheduler {
    final PriorityQueue<Task> pq;
    final SystemLogger L;
    final Stats stats;
    Scheduler(SystemLogger L, Stats stats) {
        this.L = L; this.stats = stats;
        this.pq = new PriorityQueue<>(Comparator.comparingInt((Task
t)->t.priority).thenComparing(t->t.tag==null?"":t.tag));
    }
    void enqueue(Task t) {
        pq.offer(t);
        stats.schedulerEnq++;
        L.alert(AlertType.SCHEDULER_ENQUEUE, "task=" + t);
    }
    Task dispatch() {
        Task t = pq.poll();
        if (t != null) {
            stats.schedulerDispatch++;
            L.alert(AlertType.SCHEDULER_DISPATCH, "task=" + t);
        }
        return t;
    }
}

```

```

    }
    boolean isEmpty() { return pq.isEmpty(); }
}
/* ===== Controller (pre-filters) ===== */
static final class Controller {
    final SystemLogger L;
    final int lineLen;
    Controller(SystemLogger L, int lineLen) { this.L = L; this.line
Len = lineLen; }
    int preprocessMove(int req, String id) {
        // clamp OOB and alert if needed
        if (req < 0 || req > lineLen) {
            int clamped = Math.max(0, Math.min(lineLen, req));
            L.alert(AlertType.OOB_POSITION, "AGV " + id + " request
ed OOB " + req + ", clamp->" + clamped);
            return clamped;
        }
        return req;
    }
    int compressAbsurdJump(int from, int to) {
        int d = Math.abs(to - from);
        if (d > Math.max(50, lineLen/2)) {
            // soft clamp huge jumps to 50 units at a time
            return from + (to > from ? 50 : -50);
        }
        return to;
    }
}
/* ===== Core AGV (preserves your API) ===== */
static class AGV {
    private final String id;
    private int currentPosition;
    private boolean isMoving;
    private final List<String> log;
    // Extensions:
    final Battery battery;
    final PositionSensor posSensor;
    final BatterySensor batSensor;
    final LoadSensor loadSensor;
    final Planner planner;
    final Controller controller;
    final SystemLogger L;
    final Stats stats;
    final Deque<Task> queue = new ArrayDeque<>();

```

```

    AGV(String id, int startPosition,
        Battery battery, Planner planner, Controller controller,
        SystemLogger logger, Stats stats, long sensorSeed) {
        this.id = id;
        this.currentPosition = startPosition;
        this.isMoving = false;
        this.log = new ArrayList<>();
        this.battery = battery;
        this.posSensor = new PositionSensor(this);
        this.batSensor = new BatterySensor(battery);
        this.loadSensor = new LoadSensor(sensorSeed);
        this.planner = planner;
        this.controller = controller;
        this.L = logger;
        this.stats = stats;
    }
    /* === Your original API === */
    public void moveTo(int newPosition) {
        // keep original format and behavior
        if (newPosition == currentPosition) {
            System.out.printf("AGV %s is already at Position %d.%n"
, id, newPosition);
            return;
        }
        System.out.printf("AGV %s Moving: Position %d \u2192 Positi
on %d.%n", id, currentPosition, newPosition);
        log.add(String.format("Moved from Position %d to Position %
d", currentPosition, newPosition));
        int distance = Math.abs(newPosition - currentPosition);
        currentPosition = newPosition;
        isMoving = true;
        // battery drain & stats
        battery.drainMove(distance);
        stats.moves++;
        stats.totalDistance += distance;
        // auto charge rule (simple): if low after this move, enque
ue a charge task
        if (battery.low()) {
            L.alert(AlertType.BATTERY_LOW, "AGV "+id+" battery=" +
String.format(Locale.ROOT, "%.1f", battery.level));
            queue.addLast(new Task(TaskType.CHARGE, Math.max(0, cur
rentPosition - 5), 0, "auto-charge"));
        }
    }
}

```

```

        public void stopAGV() {
            isMoving = false;
            System.out.printf("AGV %s Stopped at Position %d.%n", id, c
urrentPosition);
            log.add(String.format("Stopped at Position %d", currentPosi
tion));
            stats.stops++;
        }
        public void printLog() {
            System.out.printf("\nAGV %s Operation Log:%n", id);
            for (String logEntry : log) System.out.println(logEntry);
        }
        /* === Extended helpers === */
        public String id() { return id; }
        public int pos() { return currentPosition; }
        public double batteryLevel() { return battery.level; }
        /** Execute a planned, time-
        slotted motion (used by scheduler demo, not by your direct calls). */
        // void executePlanned(List<Long[]> schedule) {
        //     for (Long[] tp : schedule) {
        //         Long t = tp[0]; int p = (int)tp[1];
        //         // idle drain per tick
        //         battery.drainIdle();
        //         // move
        //         if (p != currentPosition) {
        //             System.out.printf("AGV %s Moving: Position %d \u219
        2 Position %d.%n", id, currentPosition, p);
        //             Log.add(String.format("Moved from Position %d to Po
        sition %d [t=%d]", currentPosition, p, t));
        //             int d = Math.abs(p - currentPosition);
        //             currentPosition = p;
        //             isMoving = true;
        //             battery.drainMove(d);
        //             stats.moves++;
        //             stats.totalDistance += d;
        //         }
        //         // Low-battery check
        //         if (battery.low()) {
        //             L.alert(AlertType.BATTERY_LOW, "AGV "+id+" Low afte
        r t="+t+" (level="+String.format(Locale.ROOT, "%.1f", battery.level)+")"
        );
        //             stats.batteryCharges++;
        //             // instant top-up for demo
        //             battery.chargeTo(100);

```

```

//      }
//    }
//  }
//
//      /** Queue a task for the AGV (used by scheduler demo). */
//      void enqueue(Task t) { queue.addLast(t); }
//
//      /** One step of internal task processing (for demo scheduler).
//      */
//      void tick(long globalTime) {
//          battery.drainIdle();
//          if (queue.isEmpty()) return;
//          Task task = queue.peekFirst();
//          if (task.type == TaskType.CHARGE) {
//              // charge spot: simply recharge and finish
//              battery.chargeTo(100);
//              L.log("AGV " + id + " charged to 100% at pos=" + currentPosition + " by " + task);
//              stats.batteryCharges++;
//              queue.pollFirst();
//              return;
//          }
//          if (task.type == TaskType.MOVE) {
//              int target = controller.preprocessMove(task.targetPos, id);
//              target = controller.compressAbsurdJump(currentPosition, target);
//              List<Integer> path = planner.plan(currentPosition, target);
//              List<long[]> sched = planner.reserve(path, globalTime, id);
//              executePlanned(sched);
//              queue.pollFirst();
//          }
//      }
//  }
// }

void executePlanned(List<long[]> schedule) {
    for (long[] tp : schedule) {
        long t = tp[0]; int p = (int)tp[1];
        // idle drain per tick
        battery.drainIdle();
        // move
        if (p != currentPosition) {

```

```

        System.out.printf("AGV %s Moving: Position %d \u2192 Position %d\n", id, currentPosition, p);
        log.add(String.format("Moved from Position %d to Position %d [t=%d]", currentPosition, p, t));
        int d = Math.abs(p - currentPosition);
        currentPosition = p;
        isMoving = true;
        battery.drainMove(d);
        stats.moves++;
        stats.totalDistance += d;
    }
    // Low-battery check
    if (battery.low()) {
        L.alert(AlertType.BATTERY_LOW, "AGV " + id + " low after t=" + t + " (level=" + String.format(Locale.ROOT, "%.1f", battery.level) + ")");
        stats.batteryCharges++;
        // instant top-up for demo
        battery.chargeTo(100);
    }
}

/** Queue a task for the AGV (used by scheduler demo). */
void enqueue(Task t) { queue.addLast(t); }
/** One step of internal task processing (for demo scheduler). */
void tick(long globalTime) {
    battery.drainIdle();
    if (queue.isEmpty()) return;
    Task task = queue.peekFirst();
    if (task.type == TaskType.CHARGE) {
        // charge spot: simply recharge and finish
        battery.chargeTo(100);
        L.log("AGV " + id + " charged to 100% at pos=" + currentPosition + " by " + task);
        stats.batteryCharges++;
        queue.pollFirst();
        return;
    }
    if (task.type == TaskType.MOVE) {
        int target = controller.preprocessMove(task.targetPos, id);
        target = controller.compressAbsurdJump(currentPosition, target);
    }
}

```

```

        List<Integer> path = planner.plan(currentPosition, target);
        List<long[]> sched = planner.reserve(path, globalTime, id);
        executePlanned(sched);
        queue.pollFirst();
    }
}

/* ===== Smart System (multi-AGV hub) ===== */
static final class SmartAGVHub {
    final Line line;
    final ReservationTable resv;
    final Scheduler scheduler;
    final Planner planner;
    final Controller controller;
    final SystemLogger L;
    final Stats stats;
    final List<AGV> fleet = new ArrayList<>();
    final AtomicLong clock = new AtomicLong(0);
    SmartAGVHub(Line line, SystemLogger logger, Stats stats) {
        this.line = line;
        this.L = logger;
        this.stats = stats;
        this.resv = new ReservationTable(logger, stats);
        this.scheduler = new Scheduler(logger, stats);
        this.planner = new Planner(line, resv, logger, stats);
        this.controller = new Controller(logger, line.length);
    }
    AGV newAGV(String id, int startPos, long seed) {
        Battery bat = new Battery(100, 0.1, 0.05);
        AGV a = new AGV(id, line.clamp(startPos), bat, planner, controller, L, stats, seed);
        fleet.add(a);
        return a;
    }
    void enqueue(Task t) { scheduler.enqueue(t); }
    /** Simple demo dispatcher: assign tasks round-robin to AGVs. */
    void dispatch() {
        int n = fleet.size();
        if (n == 0) return;
        Task t;
        int rr = 0;

```

```

        while ((t = scheduler.dispatch()) != null) {
            AGV a = fleet.get(rr % n);
            a.enqueue(t);
            rr++;
        }
    }

    /** Run ticks for internal queued tasks. */
    void runTicks(int ticks) {
        for (int i = 0; i < ticks; i++) {
            long t = clock.getAndIncrement();
            for (AGV a : fleet) a.tick(t);
        }
    }

    /** Collision check for direct user move calls. */
    void checkDirectCollision(AGV a, int target) {
        for (AGV b : fleet) {
            if (b == a) continue;
            if (b.pos() == target) {
                L.alert(AlertType.COLLISION_RISK, "AGV " + a.id() +
" -> " + target + " conflicts with " + b.id());
            }
        }
    }

}

/* ===== Scenarios (use your sequences) ===== */
enum Mode { VT, FT, Z3, UVT }
static final class Scenario {
    final String name;
    final List<Runnable> steps = new ArrayList<>();
    Scenario(String name) { this.name = name; }
    Scenario step(Runnable r) { steps.add(r); return this; }
    void run(SystemLogger L) {
        L.log("\n=== Scenario: " + name + " ===");
        int i = 0;
        for (Runnable r : steps) {
            r.run();
            i++;
        }
        L.log("Steps executed: " + i);
    }
}

static final class ScenarioFactory {
    static Scenario VT(AGV a1, AGV a2, SmartAGVHub hub) {
        Scenario sc = new Scenario("VT");
    }
}

```

```

        // Keep your first lines; add a few direct collision warnin
gs via hub
        sc.step(()-> a1.moveTo(3));
        sc.step(()-> { hub.checkDirectCollision(a2,3); a2.moveTo(3)
;});

        sc.step(()-> a2.moveTo(7));
        sc.step(()-> a2.moveTo(8));
        int[] seq = {
            81,44,36,56,29,16,21,96,77,70,57,89,22,17,10,67,87,
52,70,77,95,85,53,
            73,93,29,62,73,12,27,69,38,17,49,84,62,41,76,33,50,
87,35,57,84,62,72,
            14,48,51,73,36,34,10,71,13,43,63,95,91,49,8,30,86,4
4,66,94,46,35,50,90,
            13,44,65,72,95,18,44,38,34,47,12,48,86,95,53,29,60,
16,70,47,66,44,26,33,65
        };
        for (int x : seq) sc.step(()-> a2.moveTo(x));
        sc.step(()-> a1.stopAGV());
        sc.step(()-> a1.printLog());
        sc.step(()-> a2.printLog());
        return sc;
    }
    static Scenario UVT(AGV a1, AGV a2, SmartAGVHub hub) {
        Scenario sc = new Scenario("UVT");
        sc.step(()-> a1.moveTo(3));
        sc.step(()-> { hub.checkDirectCollision(a2,3); a2.moveTo(3)
; });

        sc.step(()-> a2.moveTo(7));
        sc.step(()-> a2.moveTo(8));
        int[] seq = {
            3,
            3,
            7,
            8,
            81,
            44,
            36,
            56,
            29,
            16,
            21,
            96,
            77,

```

70,

57,

89,

22,

17,

10,

67,

87,

52,

70,

77,

95,

85,

53,

73,

93,

29,

62,

73,

12,

27,

69,

38,

17,

49,

84,

62,

41,

76,

33,

50,

87,

35,

57,

84,

62,

72,

14,

48,

51,

73,

36,

34,

10,

```
71,  
13,  
43,  
63,  
95,  
91,  
49,  
8,  
30,  
86,  
44,  
66,  
94,  
46,  
35,  
50,  
90,  
13,  
44,  
65,  
72,  
95,  
18,  
44,  
38,  
34,  
47,  
12,  
48,  
86,  
95,  
53,  
29,  
60,  
16,  
70,  
47,  
66,  
44,  
26,  
33,  
65  
};  
for (int x : seq) sc.step()-> a2.moveTo(x));
```

```

        sc.step()-> a1.stopAGV());
        sc.step()-> a1.printLog());
        sc.step()-> a2.printLog());
        return sc;
    }
    static Scenario Z3(AGV a1, AGV a2, SmartAGVHub hub) {
        Scenario sc = new Scenario("z3");
        sc.step()-> a1.moveTo(3));
        sc.step()-> { hub.checkDirectCollision(a2,3); a2.moveTo(3)
; });

        sc.step()-> a2.moveTo(7));
        sc.step()-> a2.moveTo(8));
        int[] seq = {
            -200,
                -270,
                -135,
                -474,
                320,
                -259,
                -471,
                172,
                210,
                402,
                262,
                -243,
                -391,
                -420,
                -225,
                55,
                238,
                -246,
                261,
                -121,
                -387,
                276,
                110,
                153,
                -141,
                -440,
                47,
                142,
                -129,
                -410,
                373,

```

-171,

194,

358,

-492,

-489,

84,

204,

70,

-322,

334,

-476,

-183,

327,

-100,

-215,

-320,

231,

-102,

-213,

-491,

75,

-147,

216,

-343,

422,

262,

161,

-94,

45,

354,

-473,

181,

-356,

-75,

463,

-219,

246,

119,

262,

251,

-168,

450,

276,

-491,

```

        -144,
        -447,
        454,
        -43,
        197,
        -210,
        248,
        337,
        -207,
        -123,
        89,
        -326,
        -238,
        -16,
        482,
        299,
        -39,
        419,
        -257,
        -468,
        -226,
        -499,
        130,
        19,
        63
    };
    for (int x : seq) sc.step(()-> a2.moveTo(x));
    sc.step(()-> a1.stopAGV());
    sc.step(()-> a1.printLog());
    sc.step(()-> a2.printLog());
    return sc;
}
static Scenario FT(SmartAGVHub hub, AGV a1, AGV a2, long seed)
{
    Scenario sc = new Scenario("FT");
    sc.step(()-> a1.moveTo(3));
    sc.step(()-> { hub.checkDirectCollision(a2,3); a2.moveTo(3)
; });
    sc.step(()-> a2.moveTo(7));
    sc.step(()-> a2.moveTo(8));
    int[] seq = {
        -1280068685,
        -1280068685,
        2,

```

[illegible]

0,
16,
0,
16,
0,
0,
0,
0,
0,
0,
63,
0,
1414812756,
1414812756,
1414812756,
1414812756,
0,
0,
13041664,
0,
-1,
-1,
-254,
0,
-254,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
8,
0,
10,
0,
157,
0,
58,
0,
24634,

```

        0,
        1,
        0,
        120,
        0,
        120,
        0,
        2013265920,
        0
    };
    for (int x : seq) sc.step()-> a2.moveTo(x));
    sc.step()-> a1.stopAGV();
    sc.step()-> a1.printLog();
    sc.step()-> a2.printLog();
    return sc;
}
}
/* ===== Runner / CLI ===== */
static final class Runner {
    final SmartAGVHub hub;
    final SystemLogger L;
    Runner(SmartAGVHub hub, SystemLogger L) { this.hub = hub; this.
L = L; }
    void run(Scenario s) {
        L.log("Fleet size=" + hub.fleet.size() + ", LineLen=" + hub
.line.length);
        s.run(L);
    }
    void post() {
        for (AGV a : hub.fleet) {
            L.log(String.format(Locale.ROOT, "AGV %s: pos=%d, batte
ry=%.1f%%", a.id(), a.pos(), a.batteryLevel()));
        }
    }
}
}
/* ===== Main ===== */
public static void main(String[] args) {
    // Defaults
    Mode mode = Mode.UVT;
    long seed = 20251104L;           // Tokyo date today
    boolean consoleLog = true;      // LOG=on/off
    int agvs = 2;                   // AGVS=2..16
    int lineLen = 100;              // LINE_LEN=...
    // Parse CLI

```

```

        for (String a : args) {
            String s = a.trim();
            if (s.startsWith("MODE=")) {
                try { mode = Mode.valueOf(s.substring(5).toUpperCase(Locale.ROOT)); } catch (Exception ignore) {}
            } else if (s.startsWith("SEED=")) {
                try { seed = Long.parseLong(s.substring(5)); } catch (Exception ignore) {}
            } else if (s.startsWith("LOG=")) {
                String v = s.substring(4).toLowerCase(Locale.ROOT);
                consoleLog = v.equals("on") || v.equals("true") || v.equals("1");
            } else if (s.startsWith("AGVS=")) {
                try { agvs = Math.max(2, Math.min(16, Integer.parseInt(s.substring(5)))); } catch (Exception ignore) {}
            } else if (s.startsWith("LINE_LEN=")) {
                try { lineLen = Math.max(20, Integer.parseInt(s.substring(9))); } catch (Exception ignore) {}
            }
        }
        SystemLogger logger = new SystemLogger();
        logger.setConsole(consoleLog);
        Stats stats = new Stats();
        Line line = new Line(lineLen);
        SmartAGVHub hub = new SmartAGVHub(line, logger, stats);
        // Build fleet (at least 2 to match your sequences)
        AGV agv1 = hub.newAGV("A1", 0, seed ^ 0xA1A1A1L);
        AGV agv2 = hub.newAGV("A2", 5, seed ^ 0xA2A2A2L);
        // More AGVs (idle) if needed
        for (int i = 3; i <= agvs; i++) hub.newAGV("A"+i, i*2 % lineLen, seed ^ (0xBEEFL + i));
        // Demo scheduled tasks (not impacting your direct move sequences)
        hub.enqueue(new Task(TaskType.MOVE, lineLen/2, 5, "warmup"));
        hub.enqueue(new Task(TaskType.MOVE, Math.max(0, lineLen/2 - 10), 5, "stage-back"));
        hub.dispatch();
        hub.runTicks(10);
        // Scenarios
        Scenario sc;
        switch (mode) {
            case VT:
                sc = ScenarioFactory.VT(agv1, agv2, hub);
                break;

```

```

        case Z3:
            sc = ScenarioFactory.Z3(agv1, agv2, hub);
            break;

        case FT:
            sc = ScenarioFactory.FT(hub, agv1, agv2, seed);
            break;

        default:
            sc = ScenarioFactory.UVT(agv1, agv2, hub);

    }
    Runner runner = new Runner(hub, logger);
    logger.log("=== Smart AGV System (Mode=" + mode + ", Seed=" + seed + ", AGVs=" + agvs + ", LineLen=" + lineLen + ") ===");
    runner.run(sc);
    runner.post();
    // Stats & Logger summaries
    stats.print(logger);
    logger.summary();
}
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 85.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c125_SmartAGVSystem	12.5 %	3	21	24
> LoadSensor	0.0 %	0	2	2
main(String[])	12.5 %	3	21	24
> ScenarioFactory	25.0 %	2	6	8
> Controller	37.5 %	3	5	8
> ReservationTable	50.0 %	1	1	2
> Task	50.0 %	2	2	4
> Planner	58.3 %	7	5	12
> Scheduler	75.0 %	3	1	4
> SystemLogger	75.0 %	6	2	8
> AGV	77.8 %	14	4	18
> SmartAGVHub	85.7 %	12	2	14
> Battery	100.0 %	2	0	2
> Runner	100.0 %	2	0	2
> Scenario	100.0 %	2	0	2

3. Vibration testing instruction coverage: 99.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c125_SmartAGVSystem.java	56.9 %	2,119	1,606	3,725
▼ c125_SmartAGVSystem	50.6 %	132	129	261
> ScenarioFactory	28.3 %	497	1,258	1,755
> Line	34.9 %	15	28	43
> LoadSensor	42.9 %	9	12	21
> main(String[])	51.2 %	132	126	258
> BatterySensor	54.5 %	6	5	11
> PositionSensor	54.5 %	6	5	11
> Controller	55.2 %	32	26	58
> Planner	67.3 %	101	49	150
> ReservationTable	75.3 %	58	19	77
> Battery	87.1 %	54	8	62
> AGV	88.5 %	439	57	496
> Scheduler	92.1 %	70	6	76
> Task	92.3 %	36	3	39
> SmartAGVHub	99.4 %	179	1	180

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c125_SmartAGVSystem.java	34.5 %	38	72	110
▼ c125_SmartAGVSystem	12.5 %	3	21	24
> LoadSensor	0.0 %	0	2	2
> Scenario	0.0 %	0	2	2
> SystemLogger	12.5 %	1	7	8
> main(String[])	12.5 %	3	21	24
> ScenarioFactory	25.0 %	2	6	8
> Controller	37.5 %	3	5	8
> AGV	44.4 %	8	10	18
> Battery	50.0 %	1	1	2
> ReservationTable	50.0 %	1	1	2
> SmartAGVHub	50.0 %	7	7	14
> Task	50.0 %	2	2	4
> Planner	58.3 %	7	5	12
> Scheduler	75.0 %	3	1	4
> Alert		0	0	0
> AlertType		0	0	0

5. Fuzz testing instruction coverage: 91.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c125_SmartAGVSystem.java	37.4 %	1,384	2,318	3,702
▼ c125_SmartAGVSystem	45.8 %	109	129	238
> Stats	8.1 %	3	34	37
> ScenarioFactory	13.7 %	240	1,515	1,755
> SystemLogger	32.0 %	40	85	125
> Line	34.9 %	15	28	43
> Scenario	40.0 %	18	27	45
> LoadSensor	42.9 %	9	12	21
> AGV	46.0 %	228	268	496
> main(String[])	46.4 %	109	126	235
> BatterySensor	54.5 %	6	5	11
> PositionSensor	54.5 %	6	5	11
> Controller	55.2 %	32	26	58
> Planner	67.3 %	101	49	150
> ReservationTable	75.3 %	58	19	77
> SmartAGVHub	82.2 %	148	32	180
> Battery	83.9 %	52	10	62

6. Z3 branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c125_SmartAGVSystem.java	34.5 %	38	72	110
▼ c125_SmartAGVSystem	12.5 %	3	21	24
> LoadSensor	0.0 %	0	2	2
> Scenario	0.0 %	0	2	2
> SystemLogger	12.5 %	1	7	8
> main(String[])	12.5 %	3	21	24
> ScenarioFactory	25.0 %	2	6	8
> Controller	37.5 %	3	5	8
> AGV	44.4 %	8	10	18
> Battery	50.0 %	1	1	2
> ReservationTable	50.0 %	1	1	2
> SmartAGVHub	50.0 %	7	7	14
> Task	50.0 %	2	2	4
> Planner	58.3 %	7	5	12
> Scheduler	75.0 %	3	1	4
> Alert		0	0	0
> AlertType		0	0	0

7. Z3 instruction coverage:82.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c125_SmartAGVSystem.java	43.4 %	1,607	2,094	3,701
▼ c125_SmartAGVSystem	45.6 %	108	129	237
> Stats	8.1 %	3	34	37
> ScenarioFactory	26.4 %	464	1,291	1,755
> SystemLogger	32.0 %	40	85	125
> Line	34.9 %	15	28	43
> Scenario	40.0 %	18	27	45
> LoadSensor	42.9 %	9	12	21
> AGV	46.0 %	228	268	496
> main(String[])	46.2 %	108	126	234
> BatterySensor	54.5 %	6	5	11
> PositionSensor	54.5 %	6	5	11
> Controller	55.2 %	32	26	58
> Planner	67.3 %	101	49	150
> ReservationTable	75.3 %	58	19	77
> SmartAGVHub	82.2 %	148	32	180
> Battery	83.9 %	52	10	62

8. Unimproved V-method branch coverage: 85.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c125_SmartAGVSystem	12.5 %	3	21	24
> LoadSensor	0.0 %	0	2	2
> main(String[])	12.5 %	3	21	24
> ScenarioFactory	25.0 %	2	6	8
> Controller	37.5 %	3	5	8
> ReservationTable	50.0 %	1	1	2
> Task	50.0 %	2	2	4
> Planner	58.3 %	7	5	12
> Scheduler	75.0 %	3	1	4
> SystemLogger	75.0 %	6	2	8
> AGV	77.8 %	14	4	18
> SmartAGVHub	85.7 %	12	2	14
> Battery	100.0 %	2	0	2
> Runner	100.0 %	2	0	2
> Scenario	100.0 %	2	0	2

9. Unimproved V-method instruction coverage: 99.4%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ c125_SmartAGVSystem.java		56.9 %	2,119	1,606	3,725
✓ c125_SmartAGVSystem		50.6 %	132	129	261
> ScenarioFactory		28.3 %	497	1,258	1,755
> Line		34.9 %	15	28	43
> LoadSensor		42.9 %	9	12	21
> main(String[])		51.2 %	132	126	258
> BatterySensor		54.5 %	6	5	11
> PositionSensor		54.5 %	6	5	11
> Controller		55.2 %	32	26	58
> Planner		67.3 %	101	49	150
> ReservationTable		75.3 %	58	19	77
> Battery		87.1 %	54	8	62
> AGV		88.5 %	439	57	496
> Scheduler		92.1 %	70	6	76
> Task		92.3 %	36	3	39
> SmartAGVHub		99.4 %	179	1	180